R2.07 TP 2

Coloration de graphes

23 janvier 2026 – B. Colombel

Sommaire

1 Coloration des sommets d'un graphes avec *sagemath* **1**

2 Algorithmes de coloration de graphe **4**
2.1 Algorithme glouton (simplifié) 4
2.2 Algorithme de Welsh-Powell 5

3 Résolution d'un Sudoku **6**

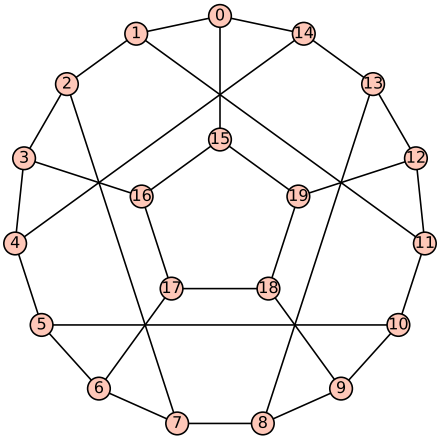
4 Fonctions utiles **7**
4.1 TP précédent 7
4.2 Couleurs prédéfinies 8

Problématique. Un problème de coloration consiste à affecter une couleur à chaque sommet d'un graphe de sorte que deux sommets voisins soient de couleurs différentes. L'objectif est de minimiser le nombre de couleurs utilisées.

1 Coloration des sommets d'un graphes avec *sagemath*

Afficher un graphe est souvent un outil précieux pour visualiser le résultat de certaines fonctions et la coloration de certains sommets, en dehors de voir les résultats du problème de coloration proprement dit, permet mettre en valeur un ensemble particulier de sommets.
Prenons en exemple le Snark Fleur, prédéfinie dans *sagemath* (voir l'article wikipedia consacré au sujet).

[1]:
```
g = graphs.FlowerSnark()
g.show()
```

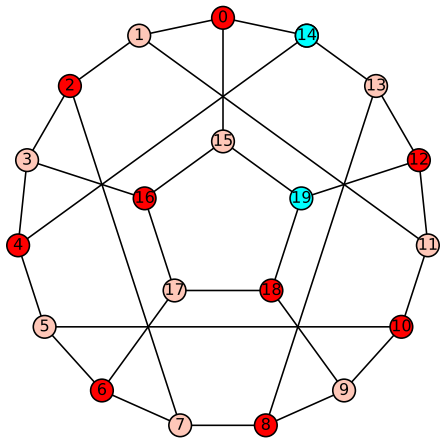


[2]:
```
g.order(), g.size()
```

[2]:
```
(20, 30)
```

L'argument `partition` de la méthode `show()` reçoit, comme son nom l'indique, une partition de l'ensemble des sommets. Une couleur est alors attribuée à chaque ensemble de la partition, afin de les séparer visuellement. Une dernière couleur est attribuée aux sommets n'apparaissant pas dans la partition. Dans notre exemple on a donc en tout trois couleurs.

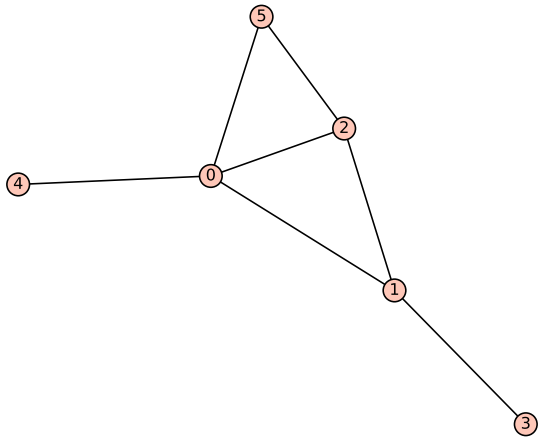
[3] :
```
S = [[0, 2, 4, 6, 8, 10, 12, 14, 16, 18], [14, 19]]
g.show(partition = S)
```



Dans la suite, un graphe sera toujours défini par sa matrice d'adjacence.

Vous pourrez tester vos fonctions avec le graphe suivant :

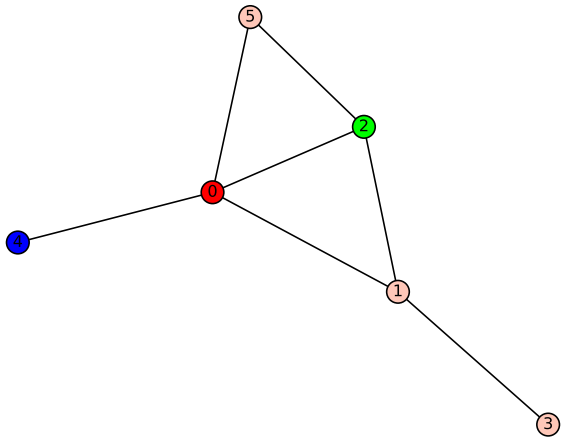
[4] :
```
M = matrix([[0,1,1,0,1,1],[1,0,1,1,0,0],[1,1,0,0,0,1],[0,1,0,0,0,0],[1,0,0,0,0,0],[1,0,1,0,0,0]])
g = Graph(M)
g.show()
```



Exercice 1. Construire une fonction `color1(M, sommets)` qui retourne une partition permettant :
- d'affecter des couleurs distinctes aux sommets donnés en paramètres.
- de colorer les autres sommets d'une autre couleur.

Exemple d'utilisation.

[7] :
```
C = color1(M, [0, 2, 4])
g.show(partition = C)
```

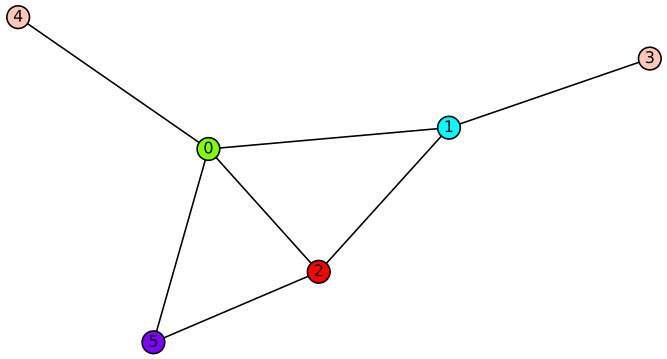


Exercice 2. Construire une fonction `color2(M, x)` qui retourne une partition permettant :
- de colorer le sommet x, ainsi que ses voisins, avec une couleur différente pour chacun d'entre eux ;
- de colorer les autres sommets d'une autre couleur.

Remarque. Penser à utiliser la fonction `voisins()` du TP précédent.

Exemple d'utilisation.

[9]:
```
P = color2(M, 2)
g.show(partition = P)
```

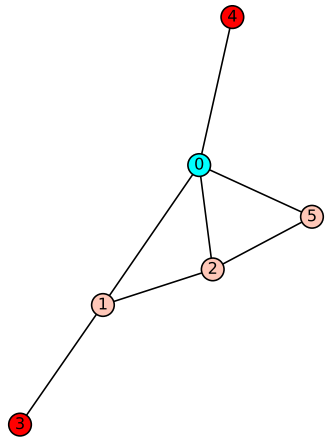


Exercice 3. Construire une fonction `color3(M)` qui retourne une liste d'entiers permettant :
- de colorer en une couleur le(s) sommet(s) de plus haut degré, et en une autre couleur celui(/ceux) de plus bas degré ;
- de colorer les autres sommets d'une autre couleur.

Remarque. Penser à utiliser la fonction `degre()` du TP précédent.

Exemple d'utilisation.

[11]:
```
C = color3(M)
g.show(partition = C)
```



2 Algorithmes de coloration de graphe

2.1 Algorithme glouton (simplifié)

Concaténer deux listes

Voici trois solutions possibles pour concaténer ou fusionner deux listes.

1. l'addition

[12]:
```
x = [10, 20,30,40]
y = [12, 14, 16, 18]
z = [55, 44, 66, 99]
opération = x + y + z
print ("Liste concaténée:\n", str (opération))
```

2. la méthode `append()`

[13]:
```
liste1 = ['A','B','C']
liste2 = ['D','E','F']
for element in liste2 :
    liste1.append(element)
print("Liste concaténée:\n",liste1)
```

3. la méthode `extend()`

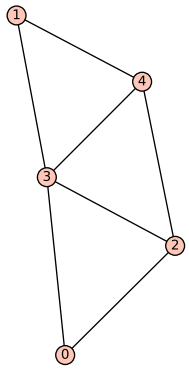
[14]:
```
liste1 = ['A','B','C']
liste2 = ['D','E','F']
liste1.extend(liste2)
print("Liste concaténée:\n",liste1)
```

Algorithme glouton (simplifié)

Entrées : S = liste des sommets d'un graphe g
1 **début**
2 | **pour** *tout* x *dans* S **faire**
3 | | pc = première couleur non utilisée par les voisins de x ;
4 | | colorer le sommet x avec pc ;
5 | **fin**
6 **fin**
Sorties : retourner la liste des couleurs de g

Exercice 4.

1. Construire la matrice d'adjacence M_1 du graphe G_1 ci-dessous :



2. Créer une fonction `glouton(M)` qui retourne une partition des sommets de g permettant de colorer un graphe de matrice M grâce à l'algorithme glouton simplifié.
3. La coloration est-elle optimale ?

2.2 Algorithme de Welsh-Powell

Tri de listes

La méthode `sort()` permet de trier une liste.

[15]:
```
L = [2,0,4,9,5]
L.sort()  # ordre croissant
```

[15]:
```
[0, 2, 4, 5, 9]
```

[16]:
```
L = [2,0,4,9,5]
L.sort(reverse=True)  # ordre déccroissant
```

[16]:
```
[0, 2, 4, 5, 9]
```

[17]:
```
L = ["AB","AAA","A","BA","B"]
L.sort()  # par ordre alphabétique
```

[17]:
```
['A', 'AAA', 'AB', 'B', 'BA']
```

La méthode `sort()` admet un paramètre optionnel `key`.
Ce paramètre prend comme argument une fonction, qui va s'appliquer sur les valeurs de la liste à trier. Le tri se fait ensuite.
Exemples d'utilisation.

[18]:
```
#tri suivant la longueur de la chaîne de caractère.
L = ["AB","AAA","A","BA","B"]
L.sort(key=len)
```

[18]:
```
['A', 'B', 'AB', 'BA', 'AAA']
```

[19]:
```
#définition de la fonction qui calcule le reste dans la division euclidienne modulo 3
def modulo3(n):
  return(n%3)

#tri suivant la valeur du reste modulo 3
```

```
L = [2,0,4,9,5]
L.sort(key=modulo3)
```

[19] :

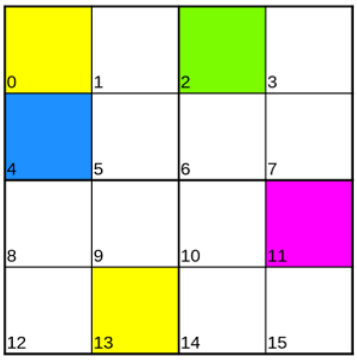
```
[0, 9, 4, 2, 5]
```

Exercice 5. Construire une fonction `tri_degre(M,Bool)` qui renvoie la liste des sommets triés suivant leur degré (dans l'ordre décroissant si `Bool = True`, croissant sinon).

Exercice 6. Écrire une fonction `glouton2(M)` qui améliorer l'algorithme glouton simplifié en classant au préalablement les sommets par ordre de degré décroissant.
La solution est-elle optimale avec G_1 ?

3 Résolution d'un Sudoku

On considère le mini-sudoku de 16 cases suivant :



On cherche à remplir ce sudoku avec 4 couleurs en considérant les contraintes habituelles :
- deux cases d'une même ligne ne peuvent pas avoir la même couleur
- deux cases d'une même colonne ne peuvent pas avoir la même couleur
- deux cases d'un même carré ne peuvent pas avoir la même couleur

Résolution

On donne ci-dessous la matrice d'adjacence `sud` du graphe à 16 sommets correspondant à une grille de mini-sudoku, sachant que deux sommets sont reliés si ils ne peuvent pas avoir la même couleur.

Exemple 1.
- le sommet 0 est relié aux sommets 1, 2, 3, 4, 5, 8 et 12 ;
- le sommet 10 est relié aux sommets 2, 6, 8, 9 ,11, 14 et 15.

[20] :

```
sud = matrix([[0,1,1,1,1,1,0,0,1,0,0,0,1,0,0,0],
              [1,0,1,1,1,1,0,0,0,1,0,0,0,1,0,0],
              [1,1,0,1,0,0,1,1,0,0,1,0,0,0,1,0],
              [1,1,1,0,0,0,1,1,0,0,0,1,0,0,0,1],
              [1,1,0,0,0,1,1,1,1,0,0,0,1,0,0,0],
              [1,1,0,0,1,0,1,1,0,1,0,0,0,1,0,0],
              [0,0,1,1,1,1,0,1,0,0,1,0,0,0,1,0],
              [0,0,1,1,1,1,1,0,0,0,0,1,0,0,0,1],
              [1,0,0,0,1,0,0,0,0,1,1,1,1,1,0,0],
              [0,1,0,0,0,1,0,0,1,0,1,1,1,1,0,0],
              [0,0,1,0,0,0,1,0,1,1,0,1,0,0,1,1],
              [0,0,0,1,0,0,0,1,1,1,1,0,0,0,1,1],
              [1,0,0,0,1,0,0,0,1,1,0,0,0,1,1,1],
              [0,1,0,0,0,1,0,0,1,1,0,0,1,0,1,1],
              [0,0,1,0,0,0,1,0,0,0,1,1,1,1,0,1],
              [0,0,0,1,0,0,0,1,0,0,1,1,1,1,1,0]])
```

Exercice 7. En modifiant l'algorithme de coloration déjà produit, construire une fonction `udoku(` ommets, couleurs)|, qui propose une coloration du sudoku, en respectant les contraintes initiales suivantes :

— `sommets` contient la liste de sommets déja colorés ;
— `couleurs` contient la liste des couleurs associées aux sommets précédents.

Exemple d'utilisation.

[22] :
```
#dans l'exemple de l'illustration ci-dessus, les sommets déja colorés sont les sommets 0,2,4,11 et 13,
↪  associés aux couleurs 1,2,3,4 et 1

sudoku([0,2,4,11,13],[1,2,3,4,1])
```

[22] :
```
[1, 4, 2, 3, 3, 2, 4, 1, 2, 3, 1, 4, 4, 1, 3, 2]
```

Exercice 8. Construire une fonction `sudoku_tableau(sommets, couleurs)` qui propose une représentation du sudoku sous forme de tableau 4 par 4.

Exemple d'utilisation.

[24] :
```
#Affichage du sudoku résolu

sudoku_tableau([0,2,4,11,13],[1,2,3,4,1])
```

[24] :
```
[1, 4, 2, 3],
[3, 2, 4, 1],
[2, 3, 1, 4],
[4, 1, 3, 2]
```

4 Fonctions utiles

4.1 TP précédent

```
def voisins(M, x) :
    p = M.ncols()
    res = []
    for j in range(p) :
        if M[x, j] == 1 :
            res.append(j)
    return res
```

```
def degre(M, x) :
    L = voisins(M, x)
    res = 0
    if x in L :
        res = len(L)+1
    else :
        res = len(L)
    return res
```

4.2 Couleurs prédéfinies

Voici la liste des 148 couleurs prédéfinies dans *sagemath* :

[30] :

```
couleurs = ['automatic', 'aliceblue', 'antiquewhite', 'aqua', 'aquamarine', 'azure', 'beige', 'bisque',
'black', 'blanchedalmond', 'blue', 'blueviolet', 'brown', 'burlywood', 'cadetblue', 'chartreuse',
'chocolate', 'coral', 'cornflowerblue', 'cornsilk', 'crimson', 'cyan', 'darkblue', 'darkcyan',
'darkgoldenrod', 'darkgray', 'darkgreen', 'darkgrey', 'darkkhaki', 'darkmagenta', 'darkolivegreen',
'darkorange', 'darkorchid', 'darkred', 'darksalmon', 'darkseagreen', 'darkslateblue', 'darkslategray',
'darkslategrey', 'darkturquoise', 'darkviolet', 'deeppink', 'deepskyblue', 'dimgray', 'dimgrey',
'dodgerblue', 'firebrick', 'floralwhite', 'forestgreen', 'fuchsia', 'gainsboro', 'ghostwhite', 'gold',
'goldenrod', 'gray', 'green', 'greenyellow', 'grey', 'honeydew', 'hotpink', 'indianred', 'indigo',
'ivory', 'khaki', 'lavender', 'lavenderblush', 'lawngreen', 'lemonchiffon', 'lightblue', 'lightcoral',
'lightcyan', 'lightgoldenrodyellow', 'lightgray', 'lightgreen', 'lightgrey', 'lightpink', 'lightsalmon',
'lightseagreen', 'lightskyblue', 'lightslategray', 'lightslategrey', 'lightsteelblue', 'lightyellow',
'lime', 'limegreen', 'linen', 'magenta', 'maroon', 'mediumaquamarine', 'mediumblue', 'mediumorchid',
'mediumpurple', 'mediumseagreen', 'mediumslateblue', 'mediumspringgreen', 'mediumturquoise',
'mediumvioletred', 'midnightblue', 'mintcream', 'mistyrose', 'moccasin', 'navajowhite', 'navy', 'oldlace',
'olive', 'olivedrab', 'orange', 'orangered', 'orchid', 'palegoldenrod', 'palegreen', 'paleturquoise',
'palevioletred', 'papayawhip', 'peachpuff', 'peru', 'pink', 'plum', 'powderblue', 'purple', 'red',
'rosybrown', 'royalblue', 'saddlebrown', 'salmon', 'sandybrown', 'seagreen', 'seashell', 'sienna',
'silver', 'skyblue', 'slateblue', 'slategray', 'slategrey', 'snow', 'springgreen', 'steelblue', 'tan',
'teal', 'thistle', 'tomato', 'turquoise', 'violet', 'wheat', 'white', 'whitesmoke', 'yellow',
'yellowgreen']

len(couleurs)
```

[30] :

```
148
```